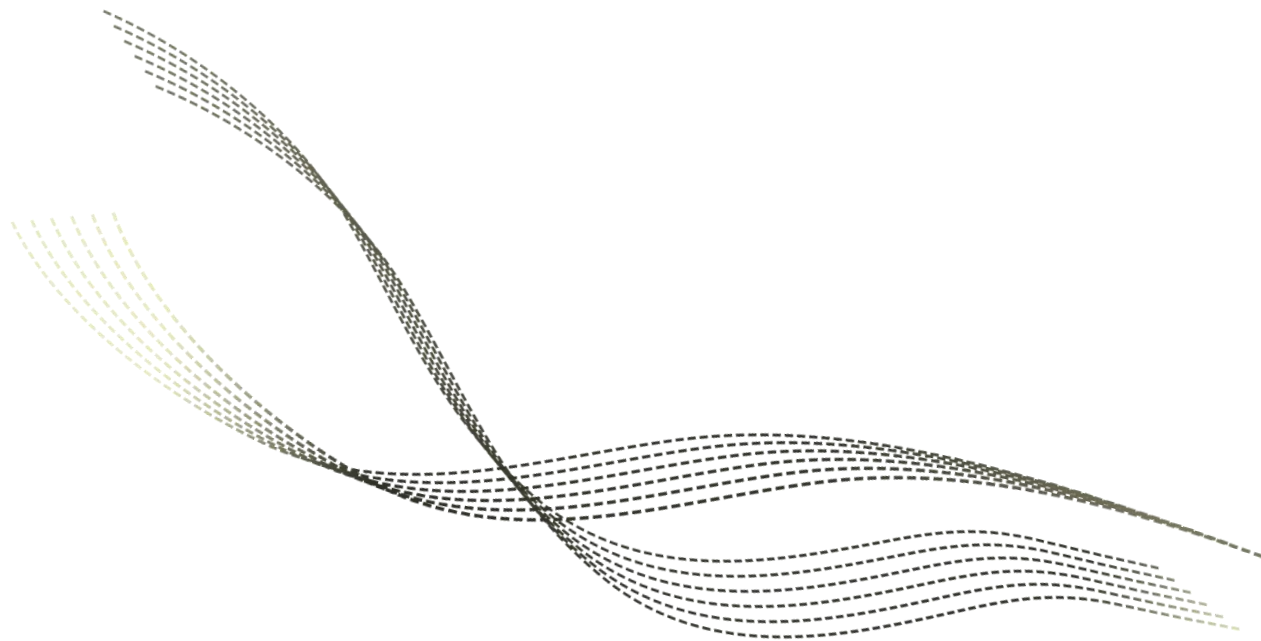


超大规模集成电路设计导论

Very Large Scale Integration Circuit (VLSI) Design

集成电路布图规划方法

沈明华



目录

CONTENTS

01

布图规划

02

贪心算法

03

搜索算法

04

线性规划

PART 03

搜索算法

■ 序列对(sequence pair)

□ 另一种解空间建模方法

- **序列对**是**非切片式矩形布图规划**的简洁表示
 - 就像波兰表达式用于切片布图规划
- 通过**一对序列**表示**非切片式布图规划**
- 可以使用**模拟退火**寻找**一个好的序列对**

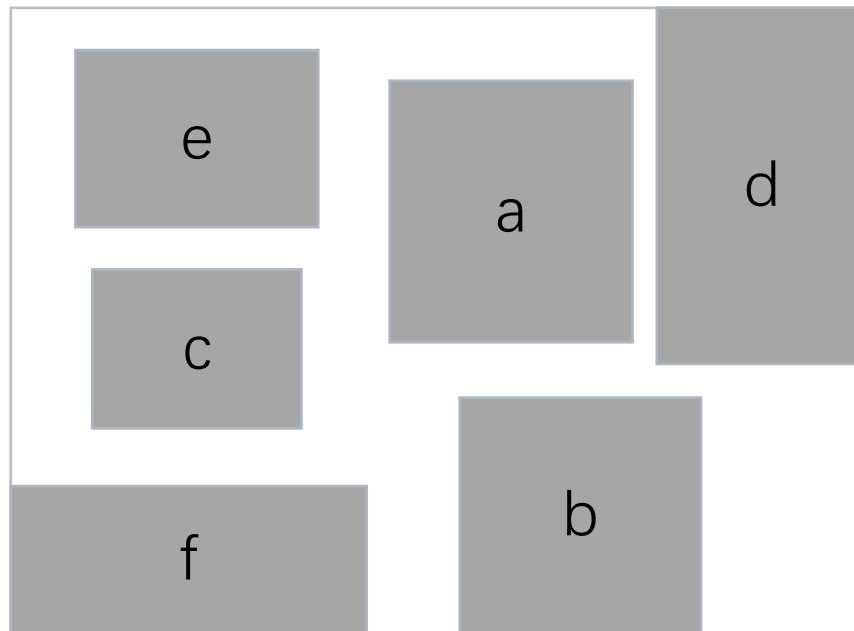
■ 序列对

□ 给定如下布图，如何用序列对来表示？

– 绘制**模块**的**正步线**(positive step line)与**负布线**(negative step line)，**正布线序列与负布线序列构成布图的序列对**

– 步线的绘制要满足三个条件：

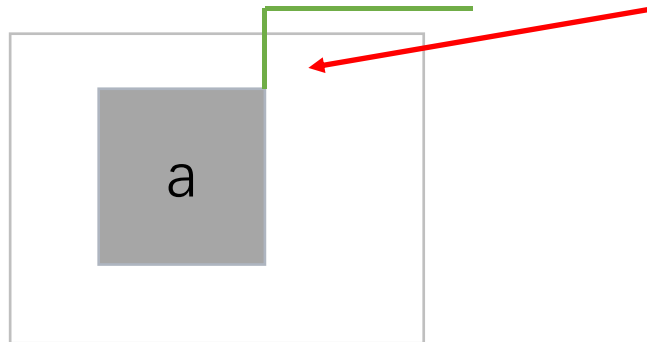
- 不穿过其他模块；
- 不与之前画的步线相交；
- 不超过芯片的边界。



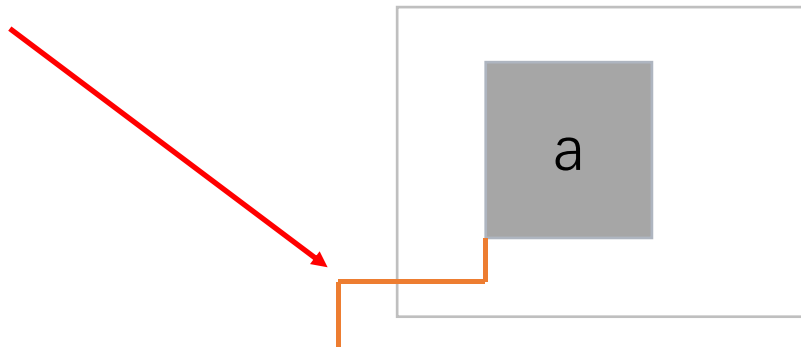
■ 序列对

□ 正步线

- 模块的**正步线**由**上右步线**，**下左步线**和**连接模块的对角线**组成。
- **上右步线**从模块的**右上角**开始，开始**向上**移动，然后**交替**向右和向上改变方向，直到到达芯片的**右上角**。



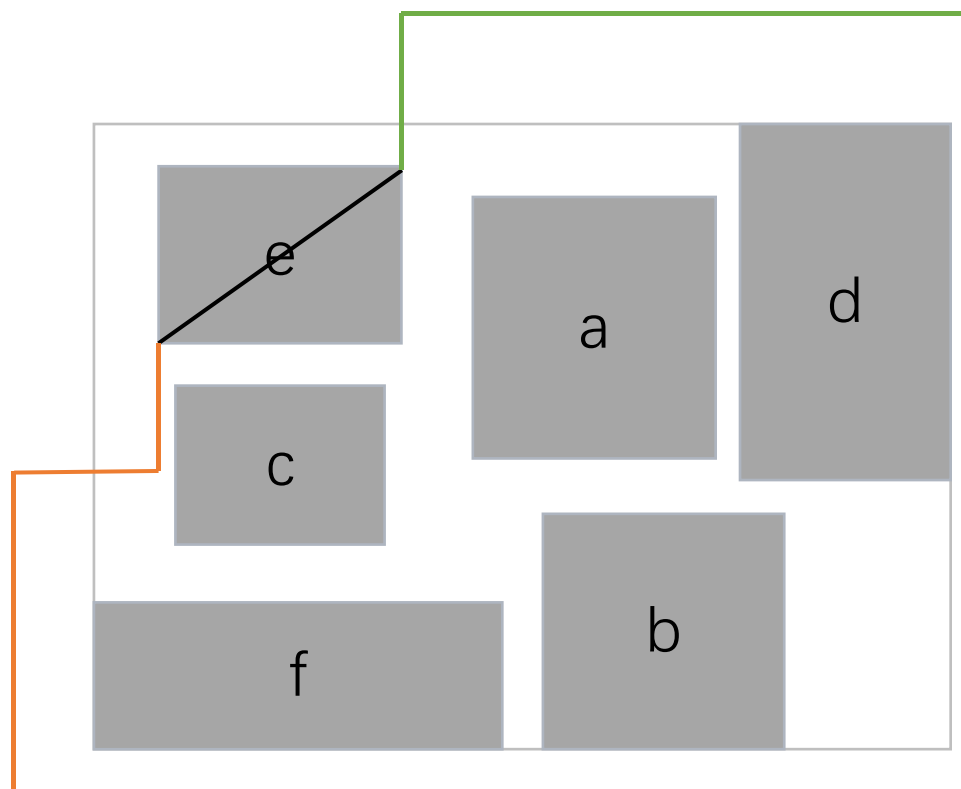
- **下左步线**从模块的**左下角**开始，开始**向下**移动，然后**交替**向左和向下改变方向，直到到达芯片的**左下角**。



■ 序列对

□ 正步线

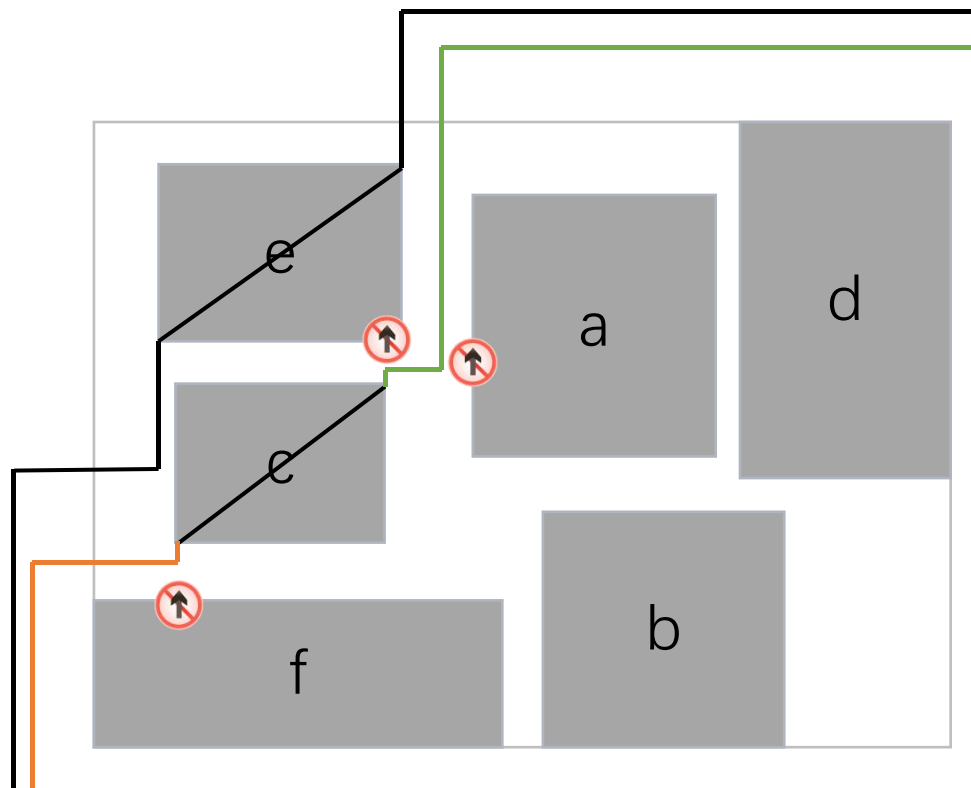
- 对于模块**e**，首先绘制模块的对角线
- 模块**e**是第一个绘制步线的模块，没有其他步线干扰
 - 步线很快就到达芯片边缘，结束绘制



■ 序列对

□ 正步线

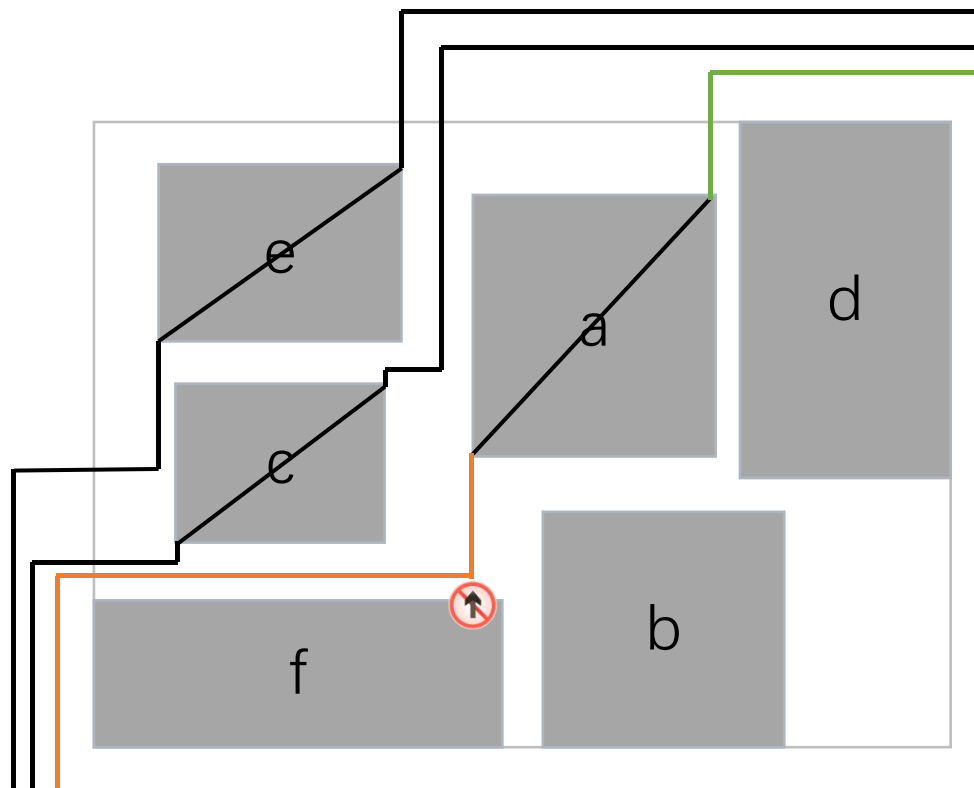
- 对于模块c，首先绘制模块的对角线
- 从右上角出发的正步线先向上走
 - 首先碰到模块e，步线绘制不能穿过其他模块，因此改变方向为向右走，又碰到模块a，再次改变方向为向上走，然后到达芯片边缘，步线绘制完毕
- 从左下角出发的正步线先向下走
 - 碰到模块f，步线不能穿过其他模块，因此改变方向为向左，到达芯片边缘，绘制完毕



■ 序列对

□ 正歩线

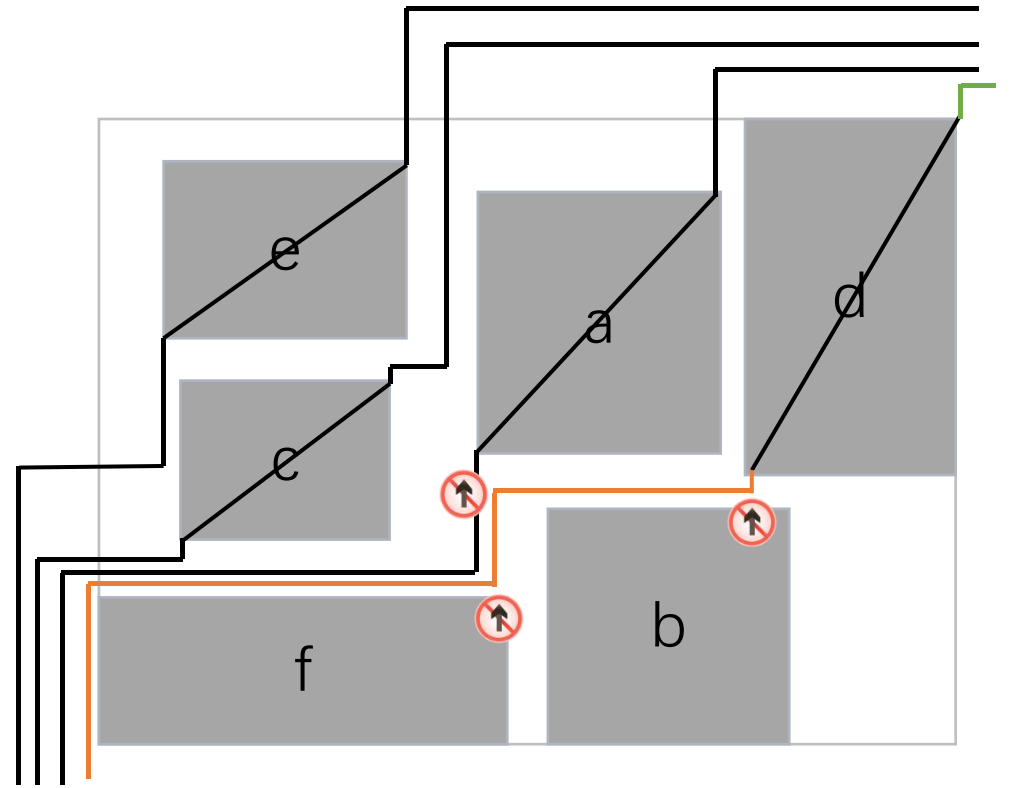
- 对于模块**a**，首先绘制模块的对角线
- 从右上角出发的正步线先向上走
 - 向上直接到达芯片边缘，绘制完毕
- 从左下角出发的正步线先向下走
 - 碰到模块**f**，改为向左走，到达芯片边缘，结束绘制



■ 序列对

□ 正步线

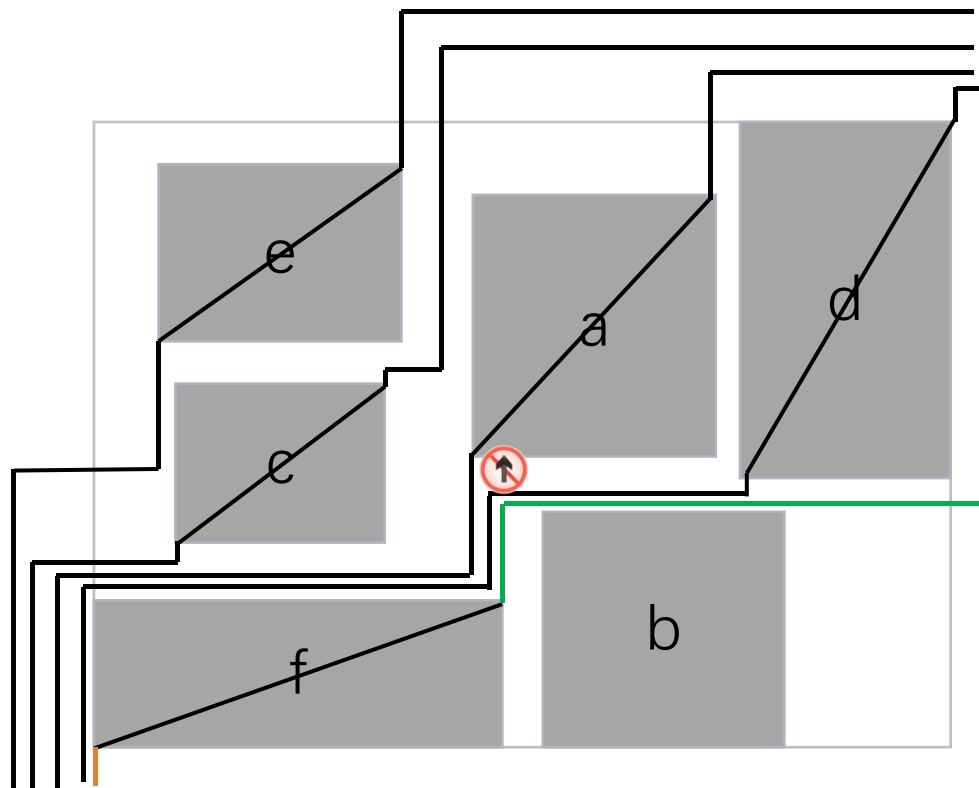
- 对于模块**d**，首先绘制模块的对角线
- 从右上角出发的正步线先向上走
 - 已经在芯片边缘，结束绘制
- 从左下角出发的正步线先向下走
 - 向下走碰到模块b，步线不能穿过模块，改为向左走
 - 随后遇到模块a的正步线，因为步线不能与步线相交，所以变为向下走
 - 遇到模块f，再次改变方向，最后到达芯片边缘



■ 序列对

□ 正步线

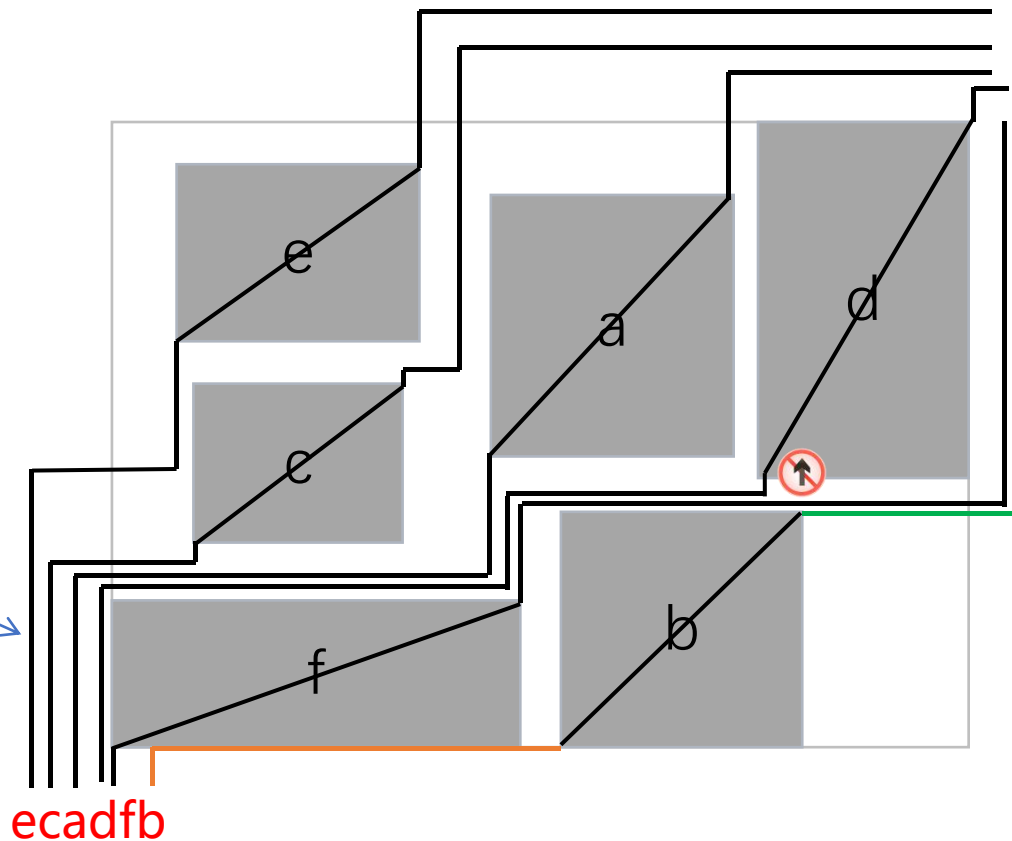
- 对于模块**f**，首先绘制模块的对角线
- 从右上角出发的正步线先向上走
 - 碰到模块d的正步线，因为步线不能相交，所以改变方向为向右，碰到芯片边缘结束绘制
- 从左下角出发的正步线先向下走
 - 到达芯片边缘，结束绘制



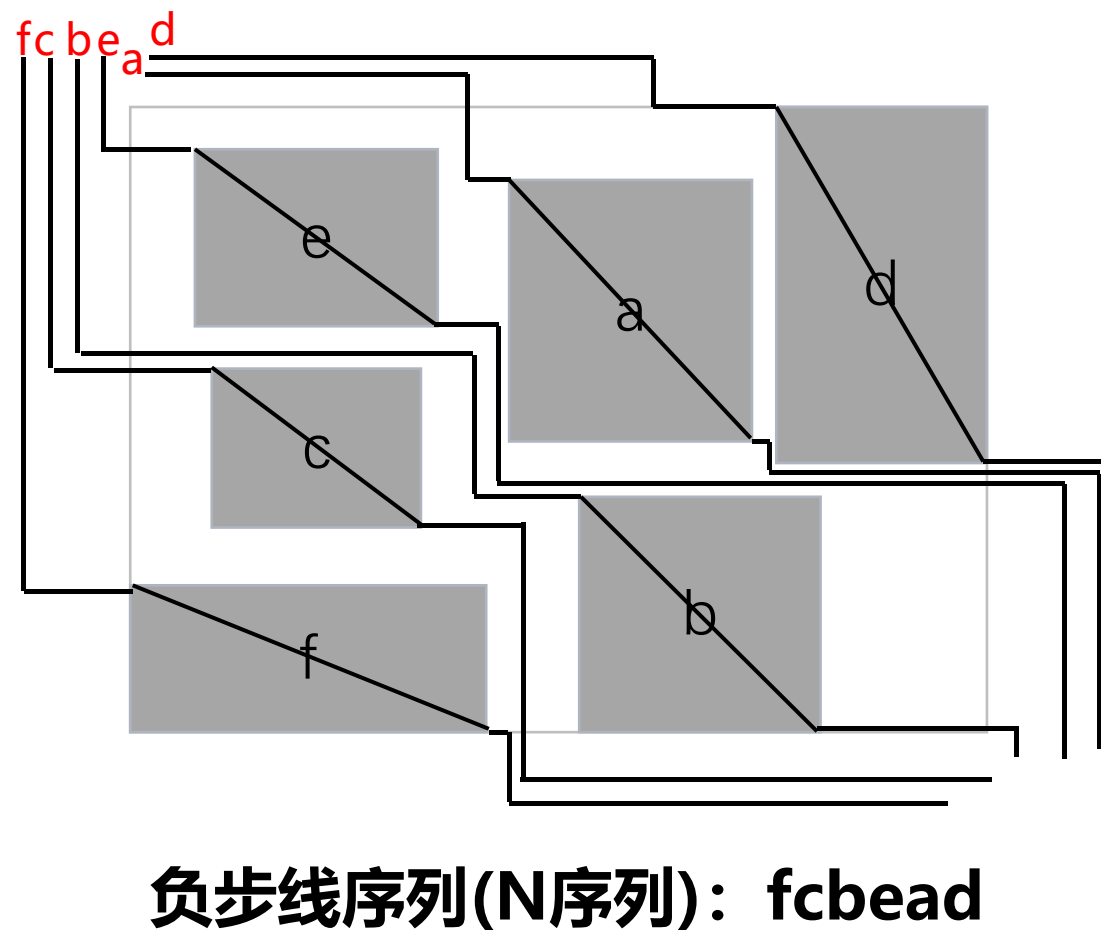
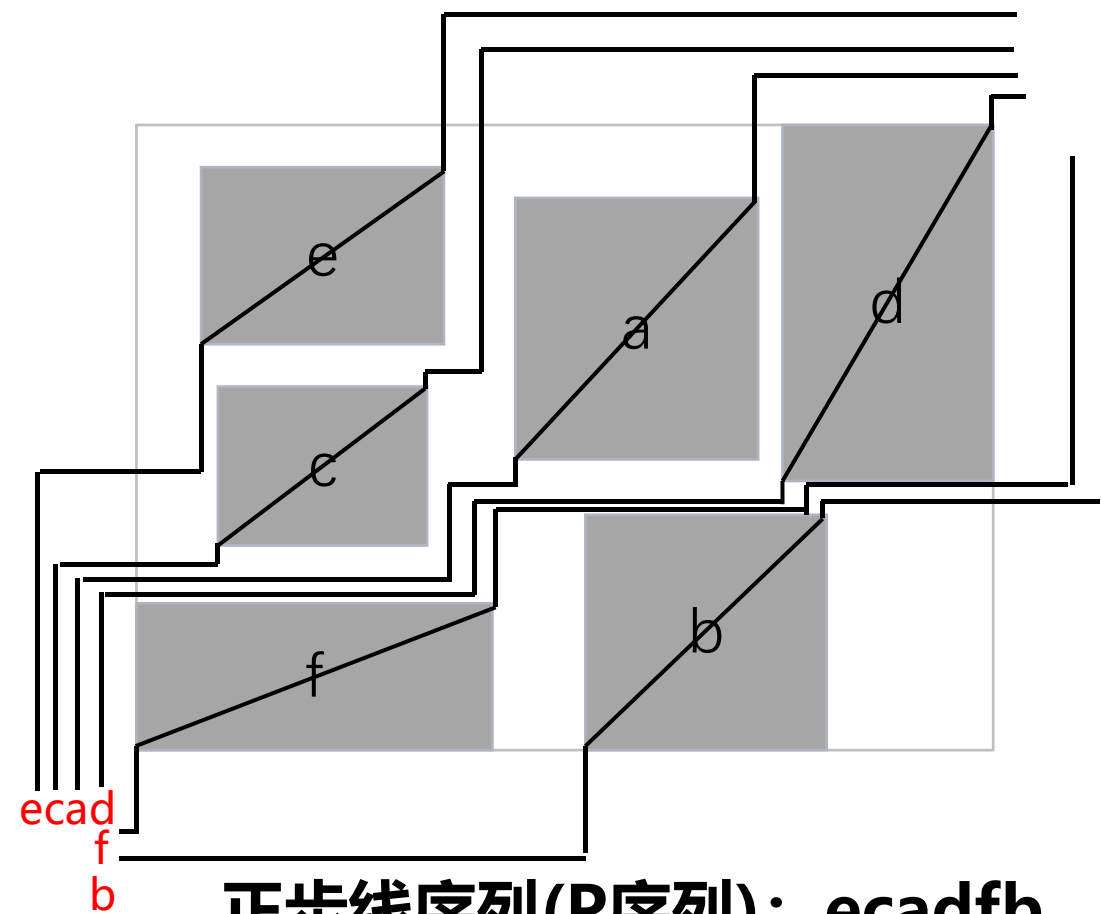
■ 序列对

□ 正步线

- 对于模块**b**，绘制模块的对角线
 - 然后按步线的绘制要求进行绘制
- 正布线序列的顺序为**芯片左下角模块**
步线从左到右，从上到下的顺序
- 最终得到正布线序列为**ecadfb**
 - 实际上，步线是在芯片边缘紧密排列的，
这里为了方便观察违反了不超过芯片边界的约束



■ 序列对

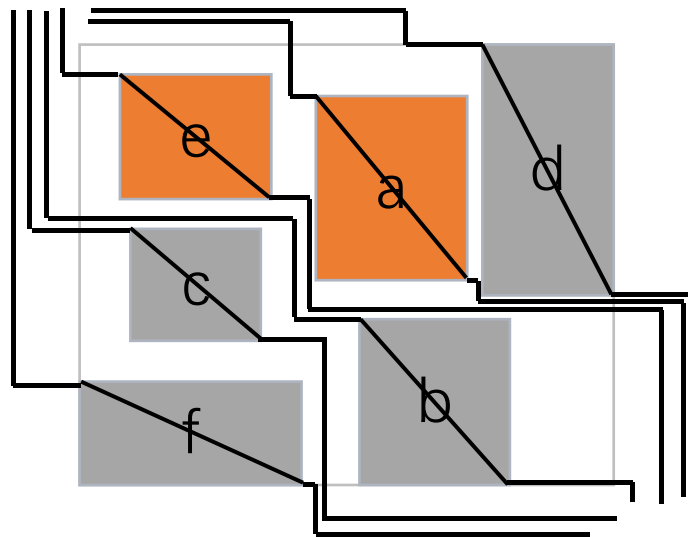
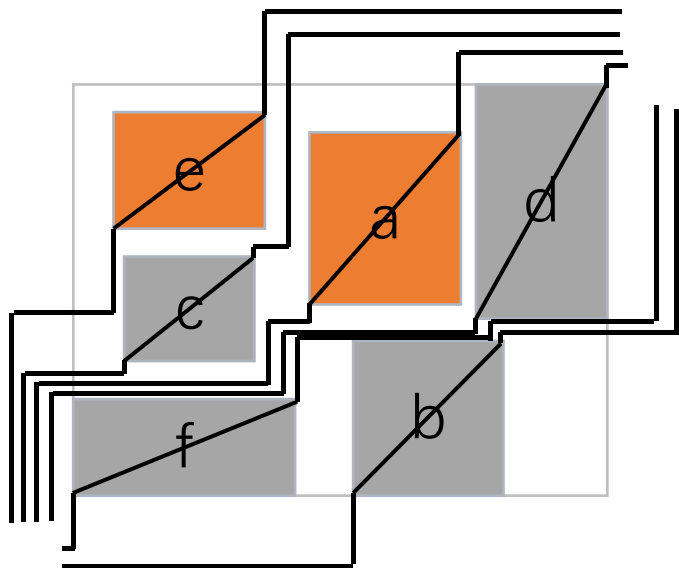


(P,N)表示布图规划的序列对

■ 序列对

□ 序列对的几何信息

- 包括四种不同相对位置关系
- **a在e的右侧**，当且仅当a在P序列和N序列中都位于e之后
- 如下图所示，模块a位于模块e的右侧

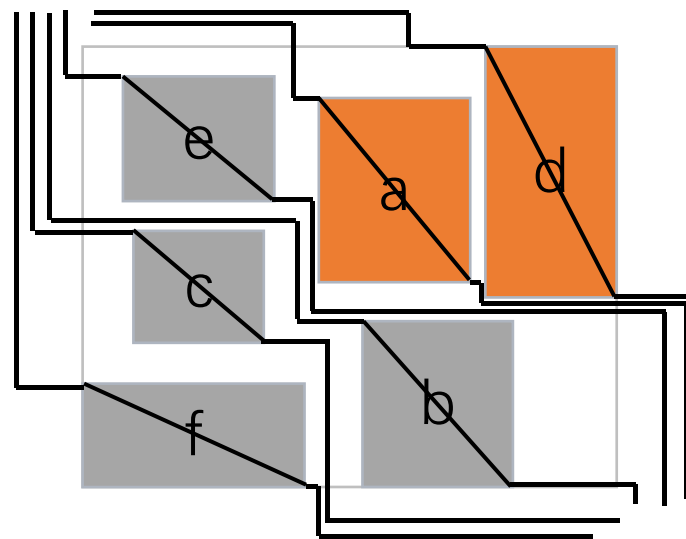
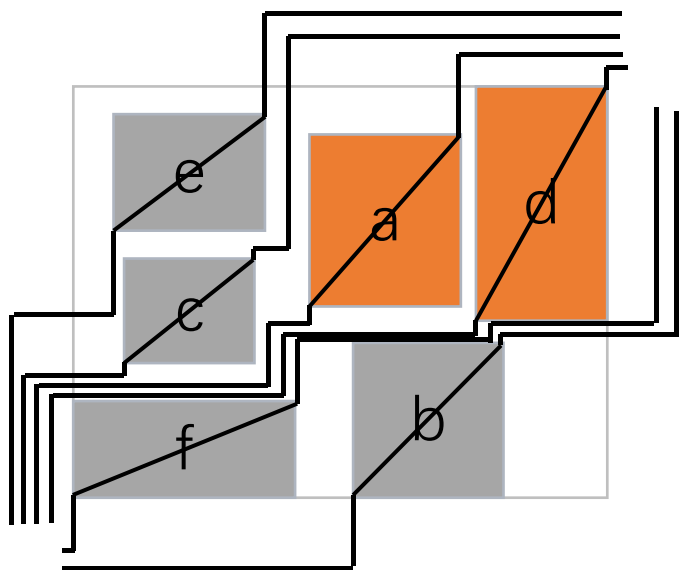


(**e**cadfb, fcb**e**ad)

■ 序列对

□ 序列对的几何信息

- **a在d的左侧**，当且仅当a在P序列和N序列中都位于d之前
- 如下图所示，模块a在模块d的左侧

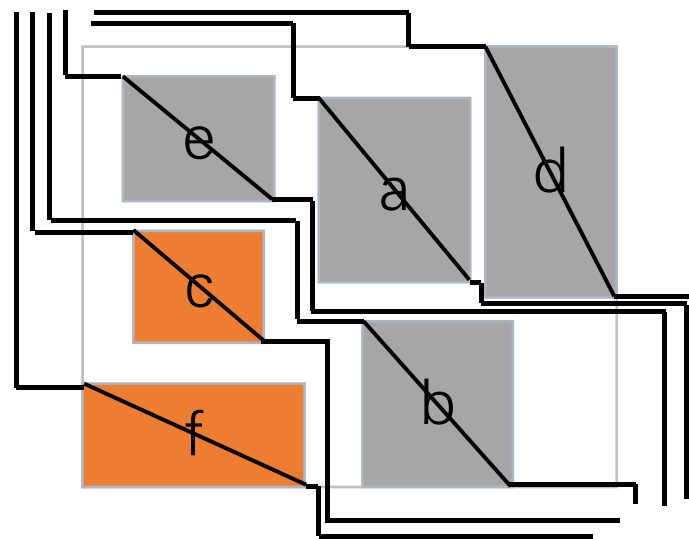
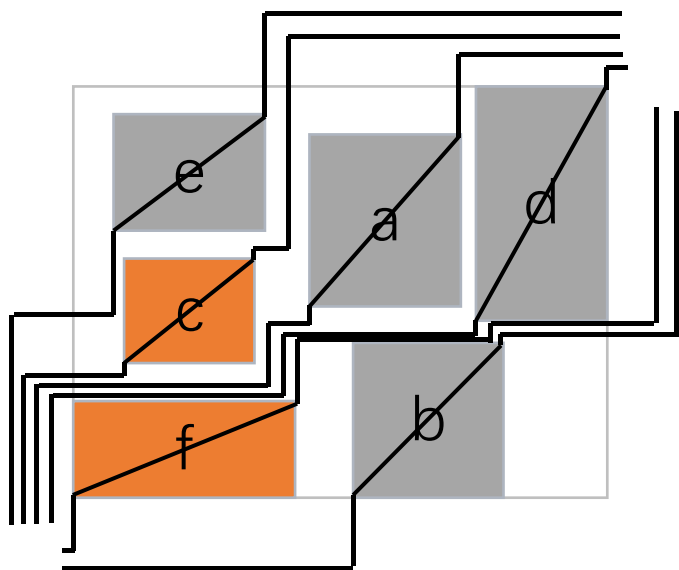


(e**cad**fb, fcb**e**ad)

■ 序列对

□ 序列对的几何信息

- **c在f的上方**，当且仅当c在P序列中位于f之前且在N序列中位于f之后
- 如下图所示，模块c位于模块f的上方

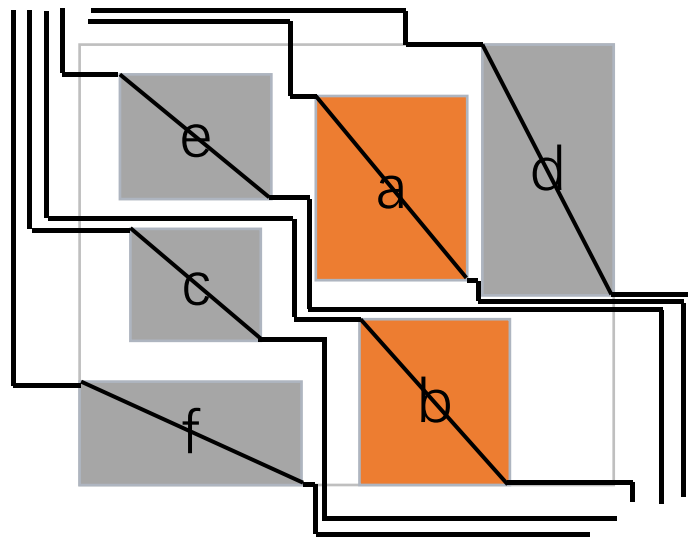
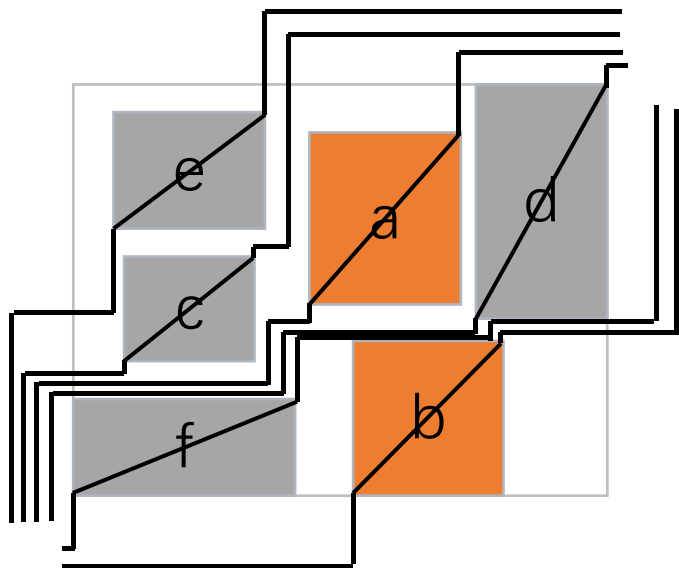


(**e**cad**f**b, **f**cbead)

■ 序列对

□ 序列对的几何信息

- **b在a的下方**，当且仅当b在P中位于a之后且在N中位于a之前
- 如下图所示，模块b位于模块a的下方

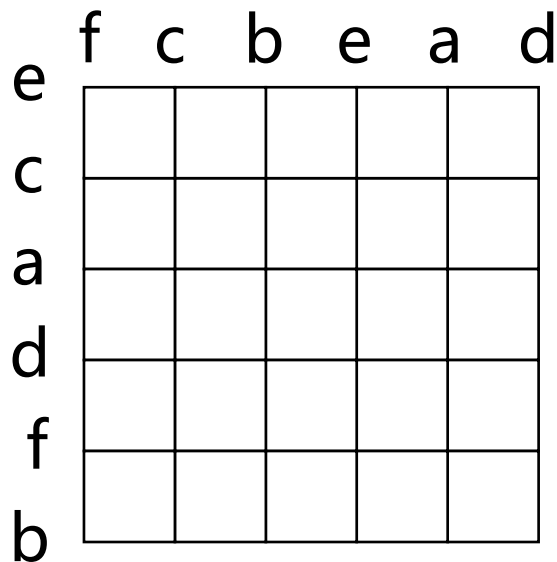


(e**a**d**f**b, f**c**e**a**d)

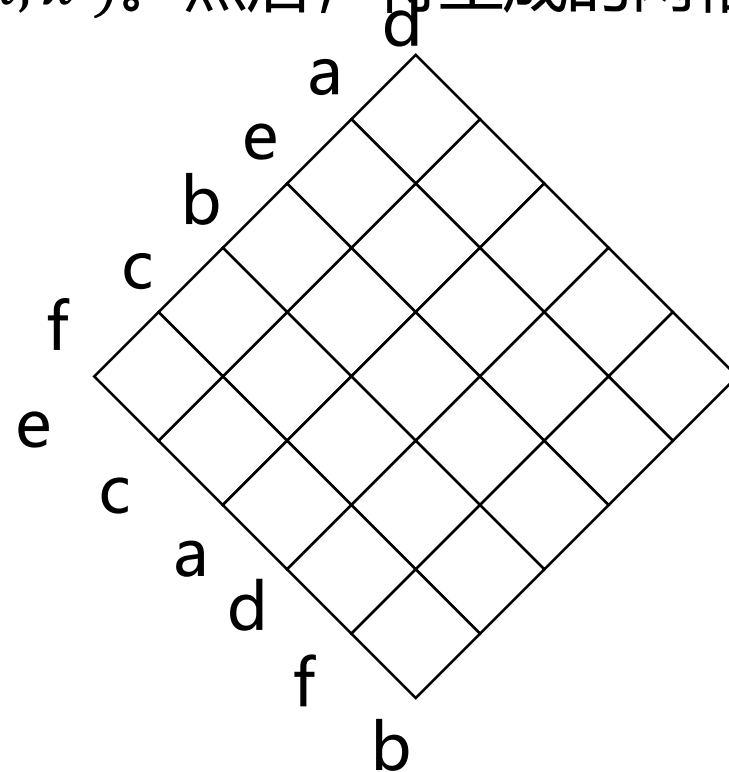
■ 序列对

□ 序列对→约束图

- 对于序列对 (ecadfb, fcbead) 和一个 $m \times m$ 的网格，水平网格线用模块名称按P序列顺序进行标记，垂直网格线用模块名称按N序列顺序进行标记
- 水平网格线 x 和垂直网格线 x' 的交点称为 (x, x') 。然后，将生成的网格逆时针旋转45度，得到一个斜向网格



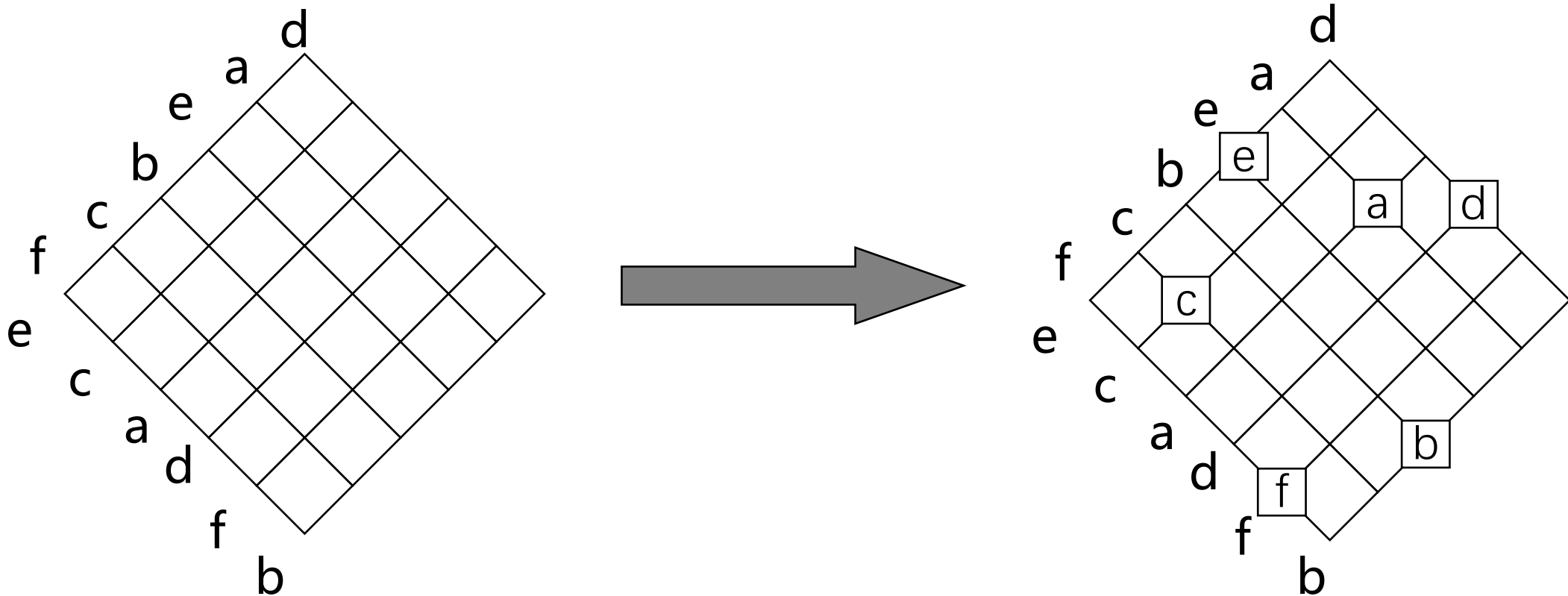
逆时针旋转45度



■ 序列对

□ 序列对→约束图

- 将**每个模块** x 的放置在 (P_x, N_x) 。
- 如有必要，扩大网格线之间的间距，以消除模块之间的重叠



■ 序列对

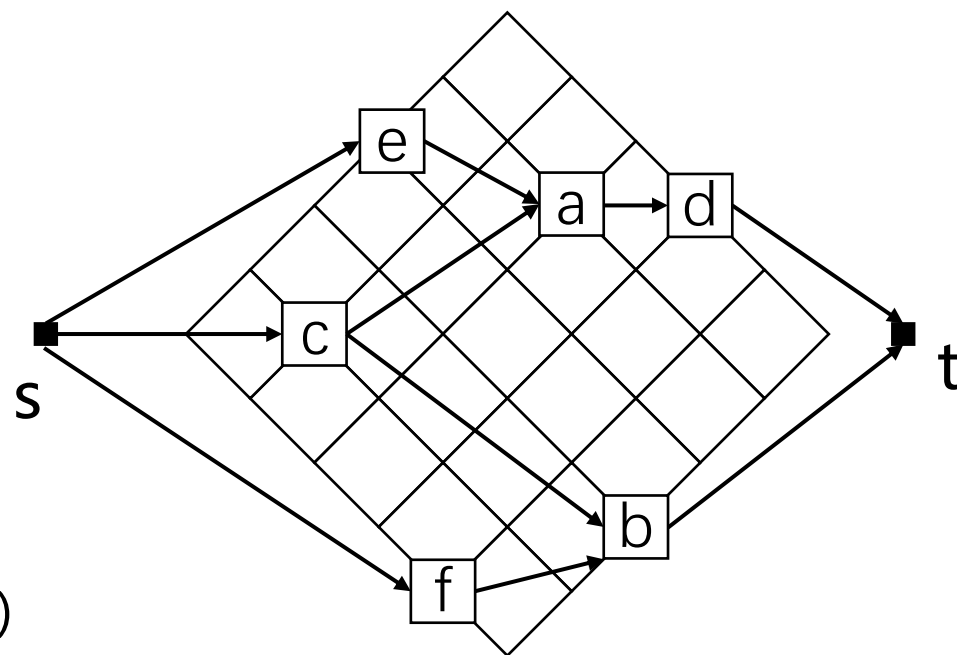
□ 构建水平约束图

1. 添加**源点** s ，**汇点** t
2. 对于**每个模块** x ，添加**边** (s, x) 和 (x, t)
3. 如果 x 在 x' 的**左侧**，则添加边 (x, x')
4. 有源点 s 和汇点 t 的边权重为0，其他边的权重为对应模块的宽度(边出发点对应模块)

(为了简化，约束图没有画出传递边；

在一个图中，如果存在 $a \rightarrow b$ ， $b \rightarrow c$ 和 $a \rightarrow c$ ，则 $a \rightarrow c$ 是传递边)

(a 在 b 的**左侧**，当且仅当 a 在P序列和N序列中都位于 b 之前)



(ecadfb, fcbead)
的水平约束图

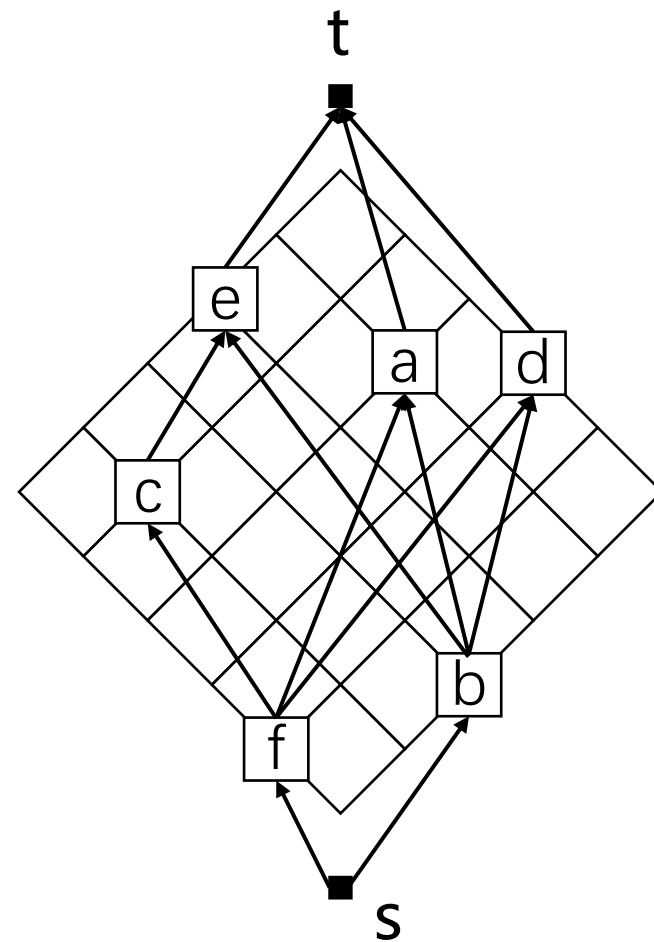
■ 序列对

□ 构建垂直约束图

1. 添加源点 s ，汇点 t
2. 对于每个模块 x ，添加边 (s, x) 和 (x, t)
3. 如果 x 在 x' 的下方，则添加边 (x, x')
4. 有源点 s 和汇点 t 的边权重为0，其他边的权重为对应模块的高度(边出发点对应的模块)

(为了简化，约束图没有画出传递边)

(a 在 b 的下方，当且仅当 a 在P序列中位于 b 之后，在N序列中位于 b 之前)



(ecadfb, fcbead)
的垂直约束图

■ 序列对

□ 总结

□ 从约束图构建布图规划结果

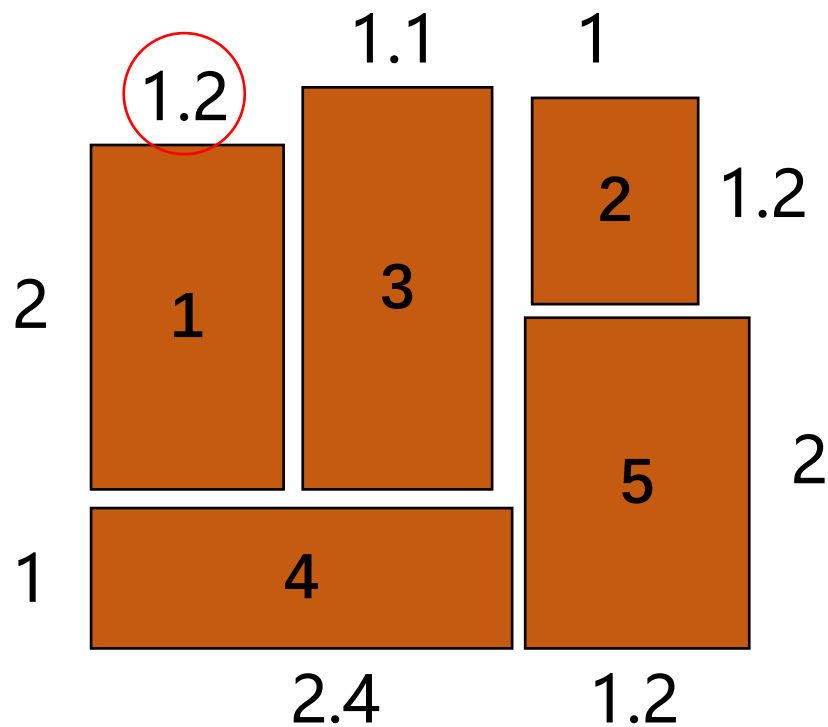
1. 每当我们在序列对中看到(...A...B..., ...A...B...), 就在水平约束图 G_h 中添加一条从 A 到 B 的边, 权重为 w_A (模块 A 的宽度)
2. 每当我们在序列对中看到(...A...B..., ...B...A...), 就在垂直约束图 G_v 中添加一条从 B 到 A 的边, 权重为 h_B (模块 B 的高度)
3. 在 G_h 和 G_v 中添加一个源点 s , 并以权重0指向所有没有入边的顶点
4. 最后, 找到从 s 到 G_h 和 G_v 中每个顶点的最长路径, 这些路径的长度即为模块在布局中左下角的坐标

■ 序列对

□ 以(13245, 41352)为例

– 下图是模块长宽部分信息

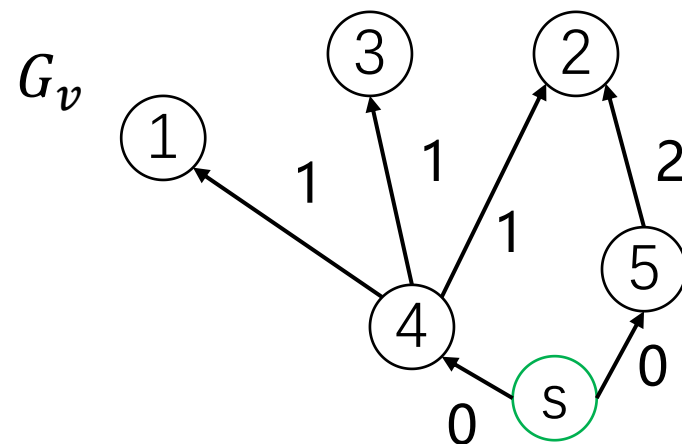
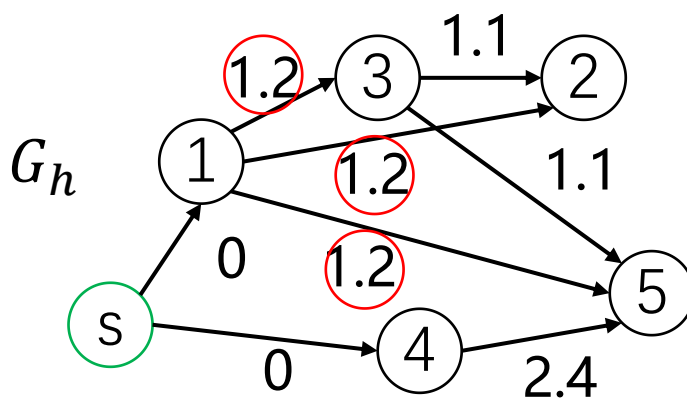
– **模块的宽度决定水平约束图的边权重**



水平约束图的边权重和出发点代表模块的宽有关
例如：

模块1的宽度是1.2，水平约束图中所有从1出发的有向边，权重都是1.2。

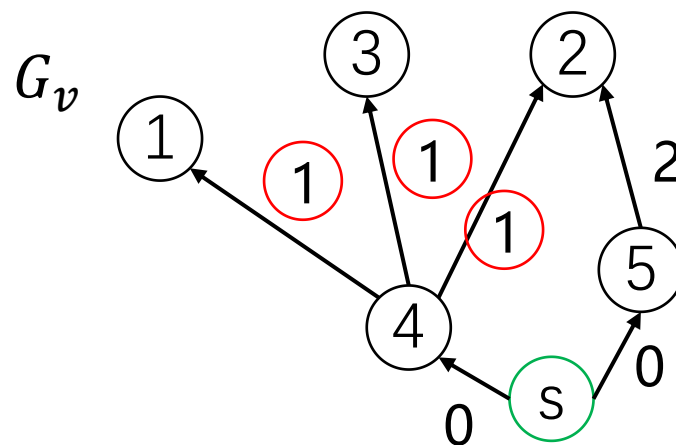
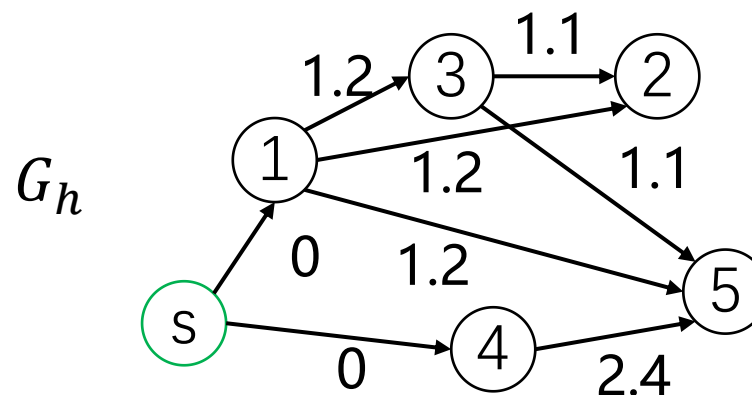
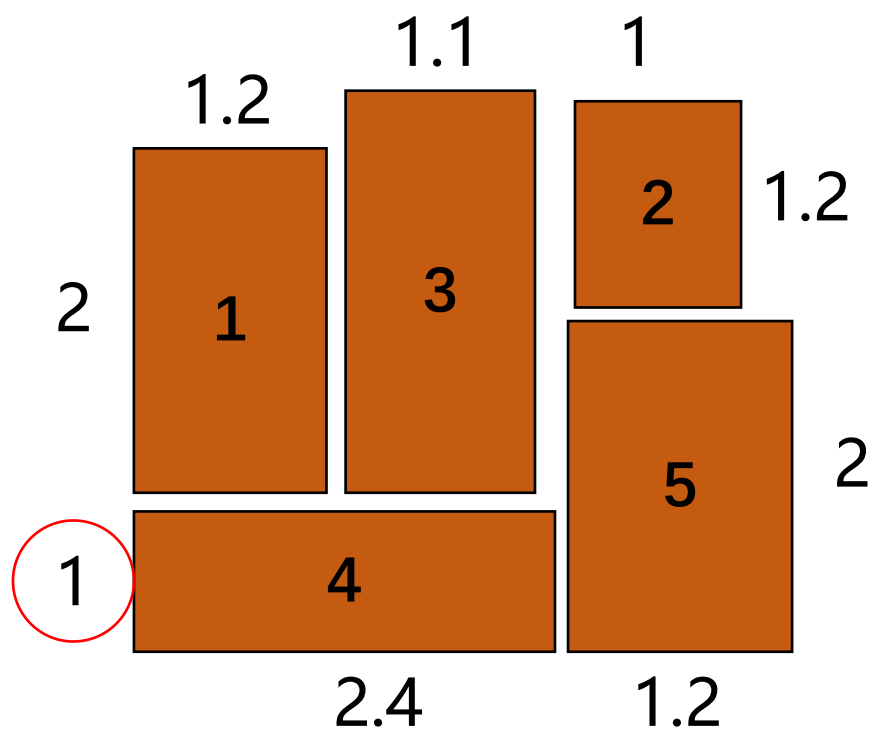
以此类推，模块4出发的有向边权重就是2.4。



■ 序列对

□ 以(13245, 41352)为例

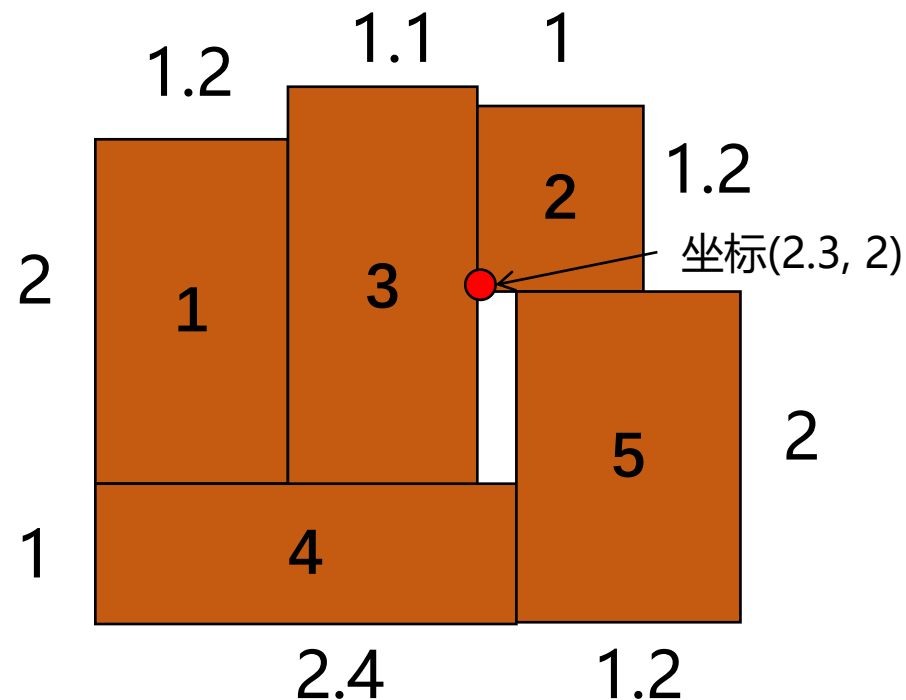
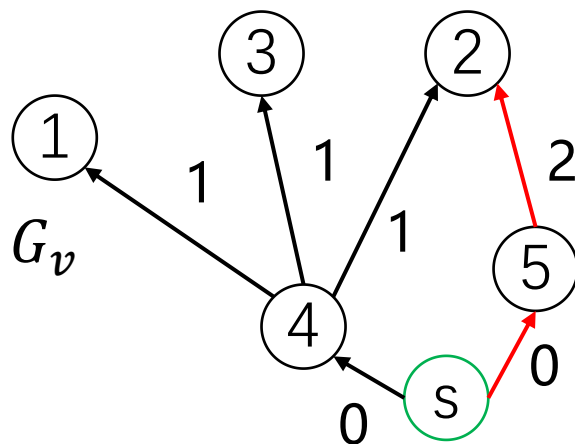
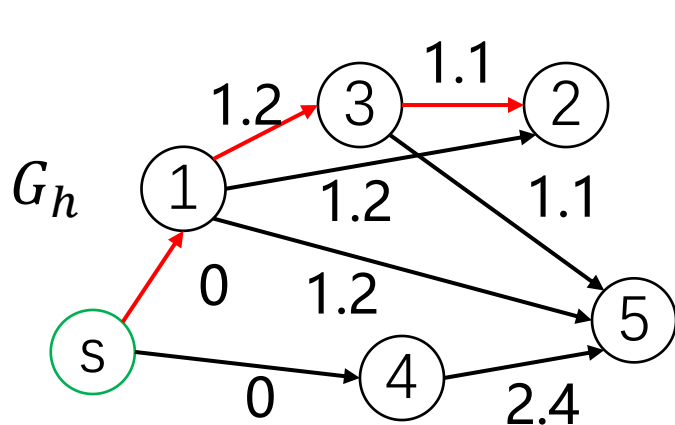
– 模块的高决定垂直约束图的边权重



■ 序列对

□ 以(13245, 41352)为例

– 根据水平和垂直约束图确定**模块的左下角坐标**



S到模块1在 G_h 图上的最长距离是0，在 G_v 图上最长距离是1，因此模块1的左下角坐标为(0,1)，对应布图中模块1的位置。

S到模块2在 G_h 图上的最长距离是2.3，在 G_v 图上最长距离是2，因此模块2的左下角坐标是(2.3, 2)

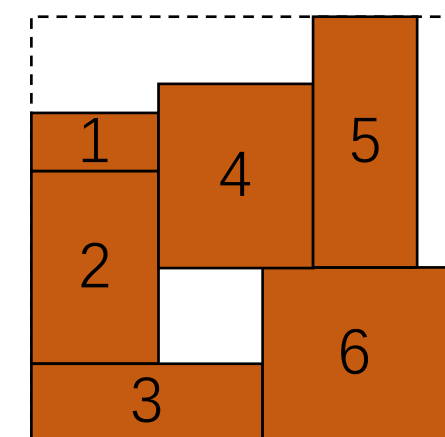
■ 序列对

□ 算法流程——模拟退火

- 初始化序列对
- 温度呈指数下降
- 在退火过程中有三种操作：
 - M1：旋转一个模块
 - M2：在两个序列中交换两个模块的位置
 - M3：在一个序列中交换两个模块的位置
- 算法让M1在低温下更可能被选择，M3在高温下更可能被选择

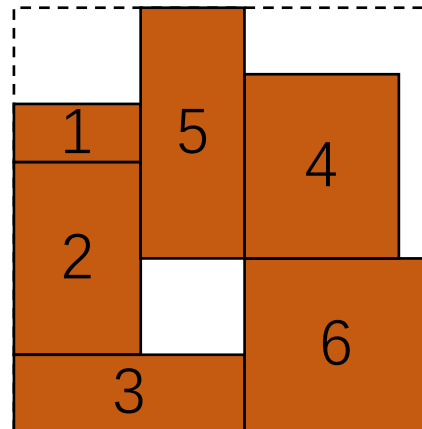
■ 序列对

□ 例子——给定序列对 (124536, 326145)



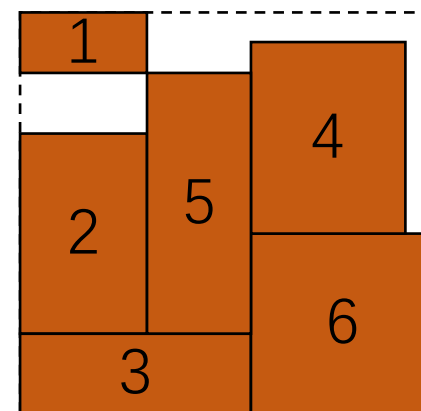
(124536, 326145)

M2
在两个序列中
交换4和5



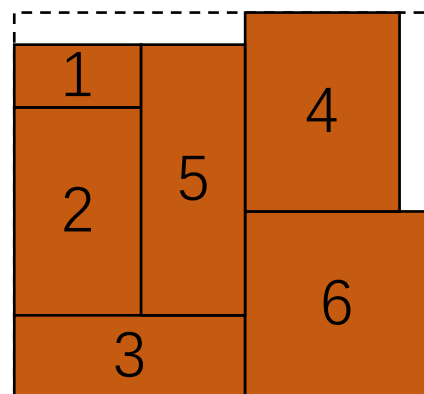
(125436, 326154)

M3
在第二个序列中
交换1和5



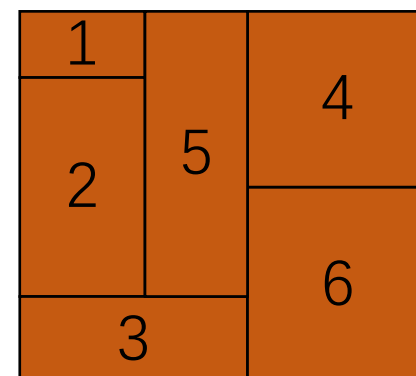
(125436, 326514)

M3
在第二个序列中
交换6和1



(125436, 321564)

M1
旋转模块4



(125436, 321564)

■ 序列对

□ 优点

- 表示简单
- 可以表示所有布图规划
- 解空间是有限的

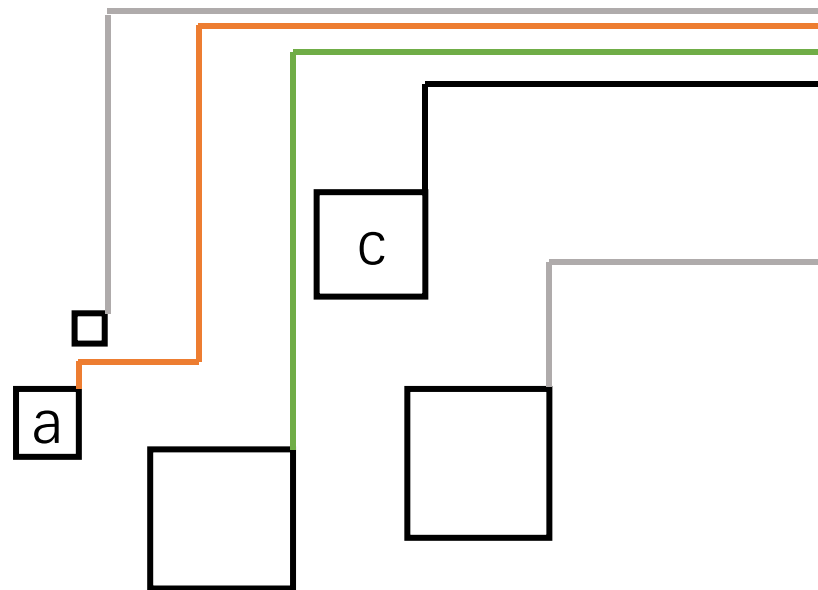
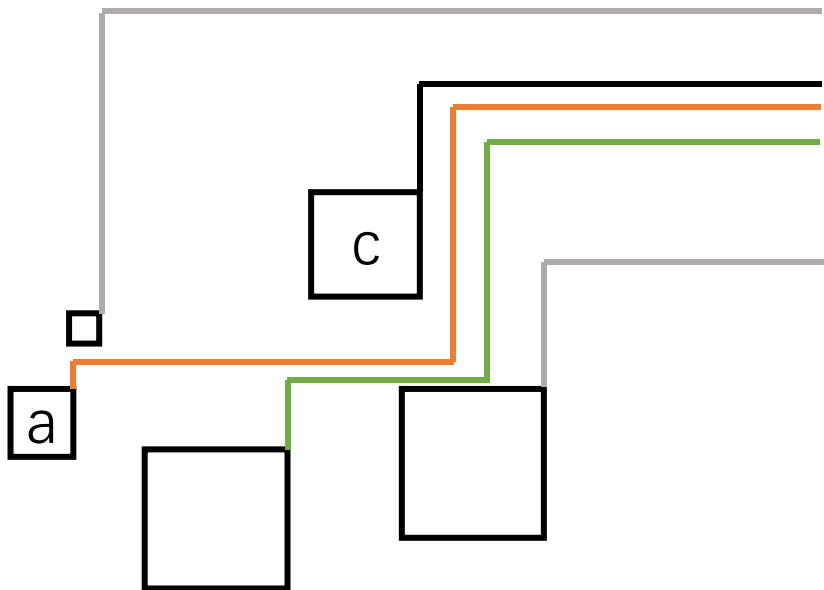
□ 缺点

- 表示存在冗余。表示不是一一对应的
- 约束图的大小为二次方，因此构建布图规划的空间复杂度是 $O(n^2)$
- 无法处理软模块

■ 序列对

□ 序列对并非唯一

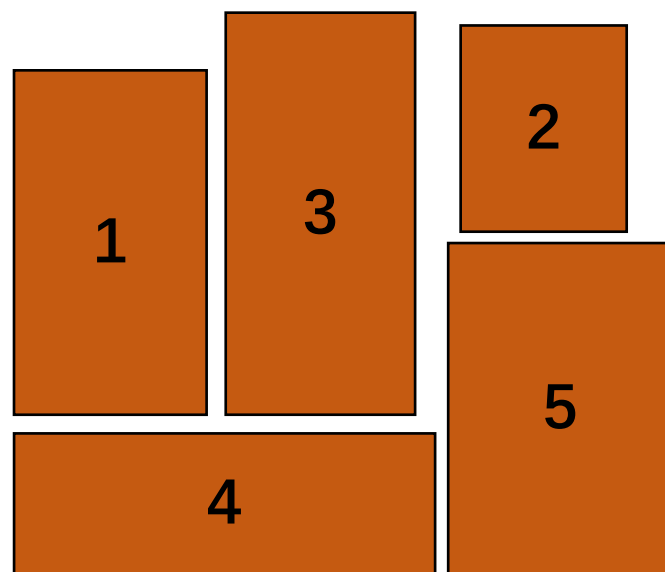
- 下图橙色线与绿色线相对黑色线的正步线序列就不唯一
- 原因是模块a在模块c的左下方
 - 既可以在步线中描述为a在c的左边，也可以描述为a在c的下边
 - 序列对**没有明确**如何表示模糊的相对位置



■ 序列对

□ 思考这个序列对：

– (13245, 41352)



□ 思考：对于这个布图，还有其他有效的序列对吗？

■ 布图规划表示

□ 建模解空间还有以下方案

– 切片

- 规范化波兰表达式: Wong & Liu [DAC-86]

– 嵌合（及一般）结构

- 角块列表 (Corner Block List) : Hong 等人 [ICCAD-00]
- Q序列 (Q-sequence) : Sakanushi & Kajitani [APCCAS-00]
- 双二元序列 (Twin Binary Sequence) : Young, Chu, Shen [ISPD-02]

– 一般结构

- 极图: Ohtsuki 等人 [ICCST-70]
- 序列对: Murata 等人 [ICCAD-95]
- 有界切片网格 (BSG) : Nakatake [ICCAD-96]
- 传递闭包图 (TCG) : Lin & Chang [DAC-01]

– 紧凑结构

- O树: Guo 等人 [DAC-99]
- B*树: Chang 等人 [DAC-00]

PART 04

线性规划

■ 线性规划

□ 数学规划，满足以下条件：

- 目标函数是线性函数
- 所有约束都是线性函数
- 部分变量是实数，部分变量是整数，即“混合整数”

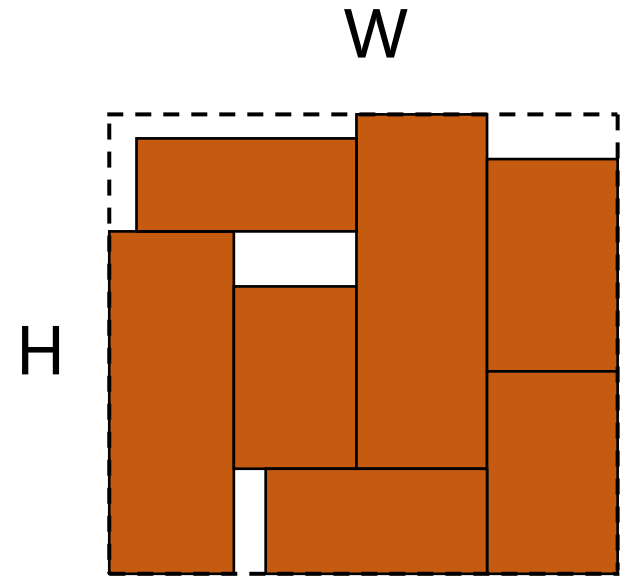
□ 混合整数线性规划^[1]，几乎类似于线性规划，只有部分变量是整数

[1] S. Sutanthavibul, E. Shragowitz, J.B. Rosen, “An analytical approach to floorplan design and optimization,” TCAD, 1991.

■ 线性规划

□ 问题定义

- 寻找合适的模块叠放方案
- 最小化布图面积——即最小化 WH
 - 但是面积 WH 并不是线性的，解决非线性问题比线性问题要困难的多，**所以我们需要把问题转为线性的**
 - **因此，我们假设 W 是固定的，最小化另一个维度 H**
- 添加约束条件，以确保模块之间不重叠
- 为每个模块 B_i 关联4个变量
 - x_i 和 y_i ：模块左下角的坐标（坐标大于或等于0）
 - w_i 和 h_i ：模块的宽度和高度

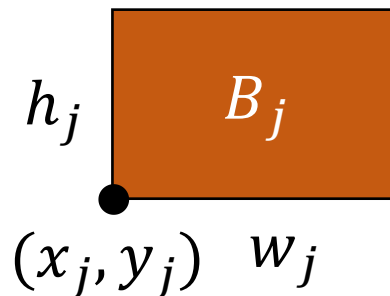
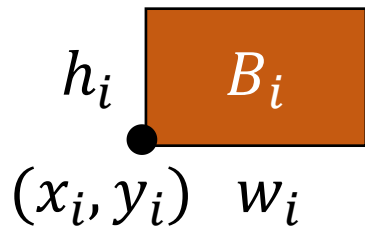


■ 线性规划

□ 非重叠约束

- 对于任意两个不重叠的模块 B_i 和 B_j , 以下4个线性约束条件中至少有一个满足:

- (1) $x_i + w_i \leq x_j$, 如果 B_i 在 B_j 的左边
- (2) $x_i - w_j \geq x_j$, 如果 B_i 在 B_j 的右边
- (3) $y_i + h_i \leq y_j$, 如果 B_i 在 B_j 的下面
- (4) $y_i - h_j \geq y_j$, 如果 B_i 在 B_j 的上面



■ 线性规划

□ 整数变量

– 使用整数变量 x_{ij} 和 y_{ij}

- 如果条件(1)成立, 则 $x_{ij} = 0$ 且 $y_{ij} = 0$
- 如果条件(2)成立, 则 $x_{ij} = 0$ 且 $y_{ij} = 1$
- 如果条件(3)成立, 则 $x_{ij} = 1$ 且 $y_{ij} = 0$
- 如果条件(4)成立, 则 $x_{ij} = 1$ 且 $y_{ij} = 1$

– 令 W 和 H 为总宽度和总高度的上限, 通常设置为芯片设计宽度和设计高度。

– 如果W和H没有给出, 则可以估算为 $W = \sum_i w_i, H = \sum_i h_i$ 。

– 非重叠约束条件改写为

(1) $x_i + w_i \leq x_j$, 如果 B_i 在 B_j 的左边

(2) $x_i - w_j \geq x_j$, 如果 B_i 在 B_j 的右边

(3) $y_i + h_i \leq y_j$, 如果 B_i 在 B_j 的下面

(4) $y_i - h_j \geq y_j$, 如果 B_i 在 B_j 的上面

$$(1') x_i + w_i \leq x_j + W(x_{ij} + y_{ij})$$

$$(2') x_i - w_j \geq x_j - W(1 + x_{ij} - y_{ij})$$

$$(3') y_i + h_i \leq y_j + H(1 - x_{ij} + y_{ij})$$

$$(4') y_i - h_j \geq y_j - H(2 - x_{ij} - y_{ij})$$

■ 线性规划

□ 整数变量

– 假设 $x_{ij} = 1, y_{ij} = 0$, 即 i 在 j 下方, 上述非重叠约束条件变为

$$(1') x_i + w_i \leq x_j + W$$

$$(2') x_i - w_j \geq x_j - 2W$$

$$(3') y_i + h_i \leq y_j$$

$$(4') y_i - h_j \geq y_j - H$$

➤ 由于模块不能超过总宽度的上限, 所以有 $x_i + w_i \leq W$, 所以(1')恒成立

➤ 由于模块不能超过总宽度的上限, 所以有 $x_j + w_j \leq W$, 可得 $x_j + w_j \leq 2W$, 所以(2')恒成立

➤ 由于模块不能超过总高度的上限, 所以有 $y_j + h_j \leq H$, 所以(4')恒成立。

– 只需要满足 $y_i + h_i \leq y_j$, B_i 可以在 B_j 下方的任意位置

■ 线性规划

□ 问题数学定义

最小化 Y (布图的高度)

$$\begin{array}{ll} \text{s.t. } 0 \leq x_i, x_i + w_i \leq W & 1 \leq i \leq n \\ 0 \leq y_i, y_i + h_i \leq Y & 1 \leq i \leq n \\ x_i + w_i \leq x_j + W(x_{ij} + y_{ij}) & 1 \leq i < j \leq n \\ x_i - w_j \geq x_j - W(1 + x_{ij} - y_{ij}) & 1 \leq i < j \leq n \\ y_i + h_i \leq y_j + H(1 - x_{ij} + y_{ij}) & 1 \leq i < j \leq n \\ y_i - h_j \geq y_j - H(2 - x_{ij} - y_{ij}) & 1 \leq i < j \leq n \\ x_{ij} = 0 \text{ or } 1 & 1 \leq i < j \leq n \\ y_{ij} = 0 \text{ or } 1 & 1 \leq i < j \leq n \end{array}$$

■ 线性规划

□ 硬模块

- 如果模块可以**旋转**，为每个模块 B_i 使用一个 0-1 整数变量 z_i 标记：
 - 如果 B_i 处于原始方向，则 $z_i = 0$ ；如果 B_i 旋转 90° ，则 $z_i = 1$

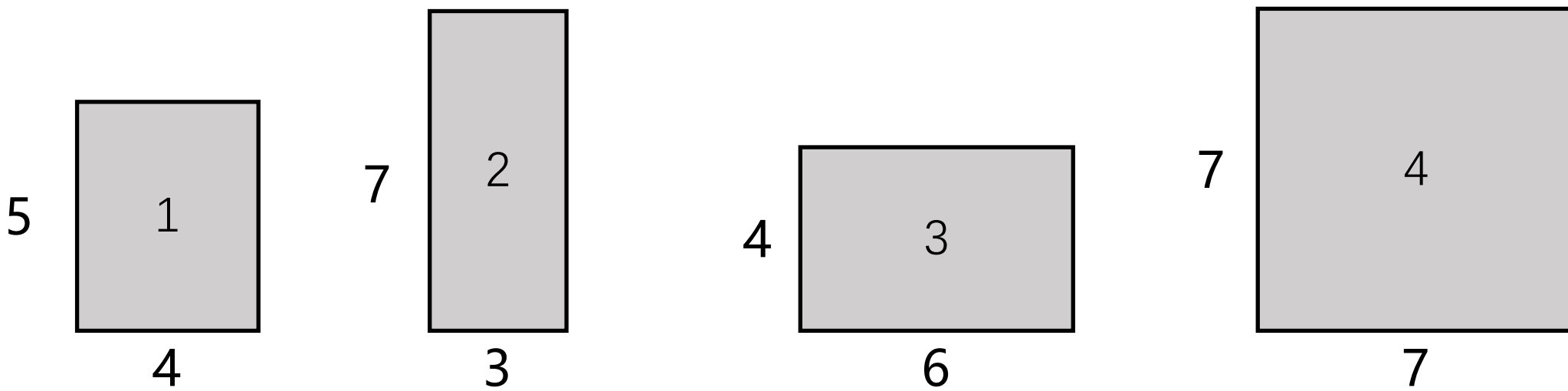
最小化 Y (布图的高度)

$$\begin{array}{ll} \text{s.t. } 0 \leq x_i, x_i + z_i h_i + (1 - z_i) w_i \leq W & 1 \leq i \leq n \\ 0 \leq y_i, y_i + z_i w_i + (1 - z_i) h_i \leq Y & 1 \leq i \leq n \\ x_i + z_i h_i + (1 - z_i) w_i \leq x_j + W(x_{ij} + y_{ij}) & 1 \leq i < j \leq n \\ x_i - z_j h_j - (1 - z_j) w_j \geq x_j - W(1 + x_{ij} - y_{ij}) & 1 \leq i < j \leq n \\ y_i + z_i w_i + (1 - z_i) h_i \leq y_j + H(1 - x_{ij} + y_{ij}) & 1 \leq i < j \leq n \\ y_i - z_j w_j - (1 - z_j) h_j \geq y_j - H(2 - x_{ij} - y_{ij}) & 1 \leq i < j \leq n \\ x_{ij} = 0 \text{ or } 1 & 1 \leq i < j \leq n \\ y_{ij} = 0 \text{ or } 1 & 1 \leq i < j \leq n \\ z_i = 0 \text{ or } 1 & 1 \leq i \leq n \end{array}$$

■ 线性规划

□ 例子

- 给定四个模块的宽度和高度: $m_1(4,5), m_2(3,7), m_3(6,4), m_4(7,7)$, 且模块不可以旋转
- 由于未知 W 和 H , 所以设置 $W = \sum_i w_i = 20, H = \sum_i h_i = 23$



■ 线性规划

□ 例子 (续)

– 问题定义为:

➤ 最小化 Y (布图的高度)

➤ 约束条件为:

$$x_i + w_i \leq x_j + 20(x_{ij} + y_{ij}), \quad \forall(i, j), 1 \leq i < j \leq 4$$

$$x_i - w_j \geq x_i - 20(1 + x_{ij} - y_{ij}), \quad \forall(i, j), 1 \leq i < j \leq 4$$

$$y_i + h_i \leq y_j + 23(1 - x_{ij} + y_{ij}), \quad \forall(i, j), 1 \leq i < j \leq 4$$

$$y_i - h_j \geq y_j - 23(2 - x_{ij} - y_{ij}), \quad \forall(i, j), 1 \leq i < j \leq 4$$

$$x_i, y_i \geq 0, \quad \forall(i, j), 1 \leq i \leq 4$$

$$x_i + w_i \leq 20, \quad \forall(i, j), 1 \leq i \leq 4$$

$$y_i + h_i \leq Y, \quad \forall(i, j), 1 \leq i \leq 4$$

$$x_{ij} = 0 \text{ or } 1, \quad \forall(i, j), 1 \leq i < j \leq 4$$

$$y_{ij} = 0 \text{ or } 1, \quad \forall(i, j), 1 \leq i < j \leq 4$$

■ 线性规划

□ 如何求解？

- Python提供pulp库求解线性规划问题
- 右图解决的是如下所示线性规划问题

$$\text{最大化 } y = 2x_1 + 3x_2 - 5x_3$$

s.t.

$$x_1 + 3x_2 + x_3 \leq 12$$

$$2x_1 - 5x_2 + x_3 \geq 10$$

$$x_1 + x_2 + x_3 = 7$$

$$x_1, x_2, x_3 \geq 0$$

```
from pulp import *

# 定义最大化问题, sense定义问题类型, LpMaximize是最大化, LpMinimize是最小化
prob = LpProblem("Prob", sense=LpMaximize)

# 定义变量, lowBound为上界, upBound为下界, cat表示变量类型, 有Continuous, Integer和Binary
x1 = LpVariable('x1', lowBound=0, upBound=7, cat='Continuous')
x2 = LpVariable('x2', lowBound=0, upBound=7, cat='Continuous')
x3 = LpVariable('x3', lowBound=0, upBound=7, cat='Continuous')

# 设置目标函数
prob += 2 * x1 + 3 * x2 - 5 * x3

# 添加约束条件
prob += (2 * x1 - 5 * x2 + x3 >= 10)
prob += (x1 + 3 * x2 + x3 <= 12)
prob += (x1 + x2 + x3 == 7)

# 求解
prob.solve()

print("Status:", LpStatus[prob.status]) # 输出求解状态
for v in prob.variables():
    print(v.name, "=", v.varValue) # 输出每个变量的最优值
print("F(x) = ", value(prob.objective)) # 输出最优解的目标函数值
```

■ 线性规划

□ 例子 (续)

– 求解得到:

$$Y = 7$$

$$(x_1, y_1) = (0,0), (x_2, y_2) = (4,0), (x_3, y_3) = (7,0), (x_4, y_4) = (13,0)$$

$$(x_{12}, y_{12}) = (0,0) \text{ —— } 1 \text{ 在 } 2 \text{ 的左边}$$

$$(x_{13}, y_{13}) = (0,0) \text{ —— } 1 \text{ 在 } 3 \text{ 的左边}$$

$$(x_{14}, y_{14}) = (0,0) \text{ —— } 1 \text{ 在 } 4 \text{ 的左边}$$

$$(x_{23}, y_{23}) = (0,0) \text{ —— } 2 \text{ 在 } 3 \text{ 的左边}$$

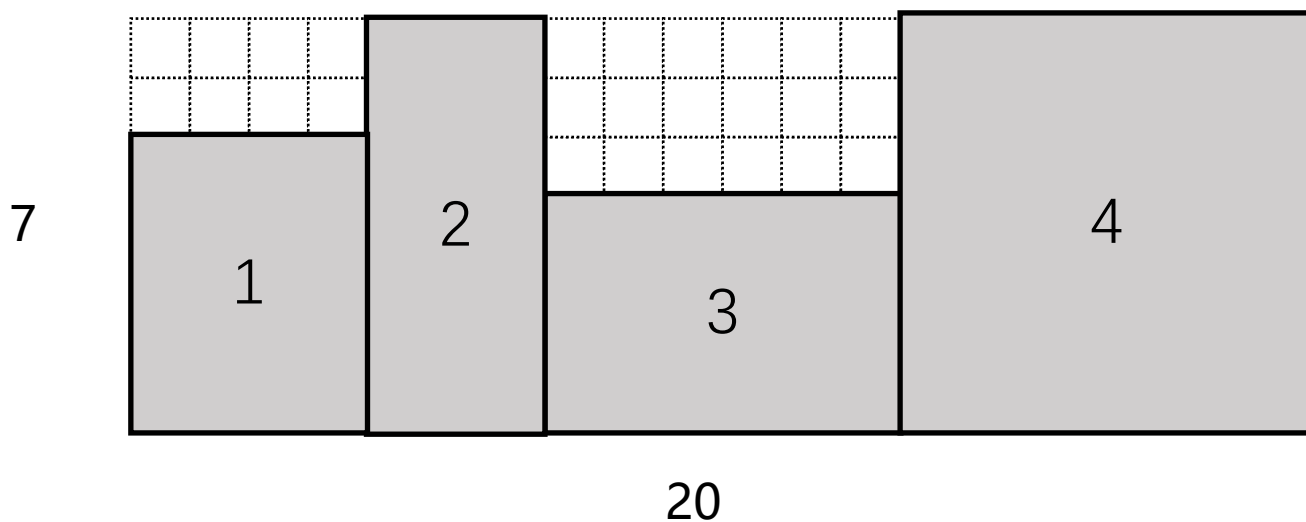
$$(x_{24}, y_{24}) = (0,0) \text{ —— } 2 \text{ 在 } 4 \text{ 的左边}$$

$$(x_{34}, y_{34}) = (0,0) \text{ —— } 3 \text{ 在 } 4 \text{ 的左边}$$

■ 线性规划

□ 例子 (续)

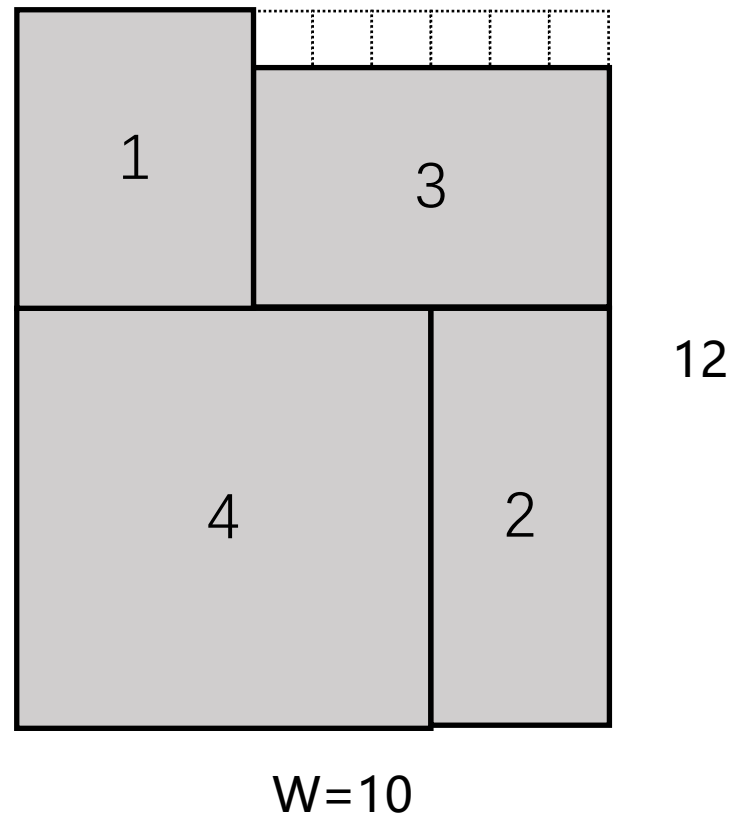
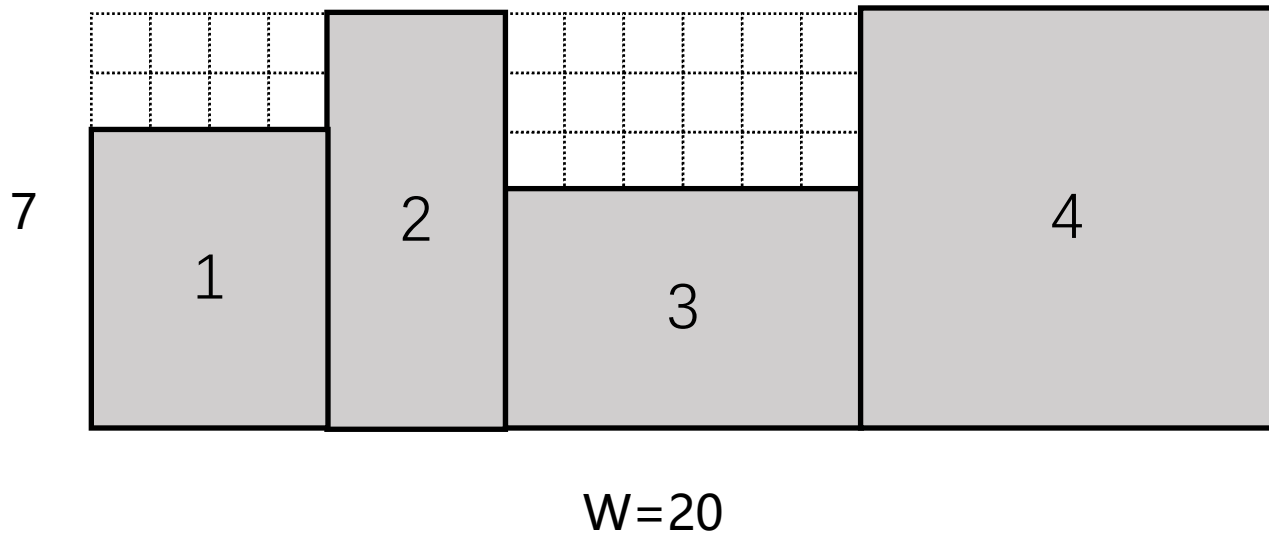
- 最终得到的结果，面积为 20×7



■ 线性规划

□ 例子 (续)

- 但是, 当我们将 W 取10时, 得到面积为 10×12 的结果, 比前面的要好
- 这说明, 结果质量受限于 W 的取值



■ 线性规划

□ 软模块

- 如果 B_i 是软模块, $w_i h_i = A_i$ 。但这个约束是二次的, 我们需要将其线性化
- 通过在 w_{imax} (模块 B_i 的最大宽度) 处对 $h_i = A_i/w_i$ 的泰勒展开式取前两项, 将其线性化

$$h_i = h_{imin} + \lambda_i(w_{imax} - w_i)$$

其中, $h_{imin} = A_i/w_{imax}$ 且 $\lambda_i = A_i/w_{imax}^2$

■ 线性规划

□ 如果 B_i 是软模块, B_j 是硬模块, 非重叠约束变成:

$$(1) x_i + w_i \leq x_j + W(x_{ij} + y_{ij})$$

$$(2) x_i - w_j \geq x_j - W(1 + x_{ij} - y_{ij})$$

$$(3) y_i + h_{imin} + \lambda_i(w_{imax} - w_i) \leq y_j + H(1 - x_{ij} + y_{ij})$$

$$(4) y_i - h_j \geq y_j - H(2 - x_{ij} - y_{ij})$$

□ 如果 B_i 是软模块, B_j 也是软模块, 非重叠约束变成:

$$(1) x_i + w_i \leq x_j + W(x_{ij} + y_{ij})$$

$$(2) x_i - w_j \geq x_j - W(1 + x_{ij} - y_{ij})$$

$$(3) y_i + h_{imin} + \lambda_i(w_{imax} - w_i) \leq y_j + H(1 - x_{ij} + y_{ij})$$

$$(4) y_i - h_{jmin} - \lambda_j(w_{jmax} - w_j) \geq y_j - H(2 - x_{ij} - y_{ij})$$

■ 线性规划

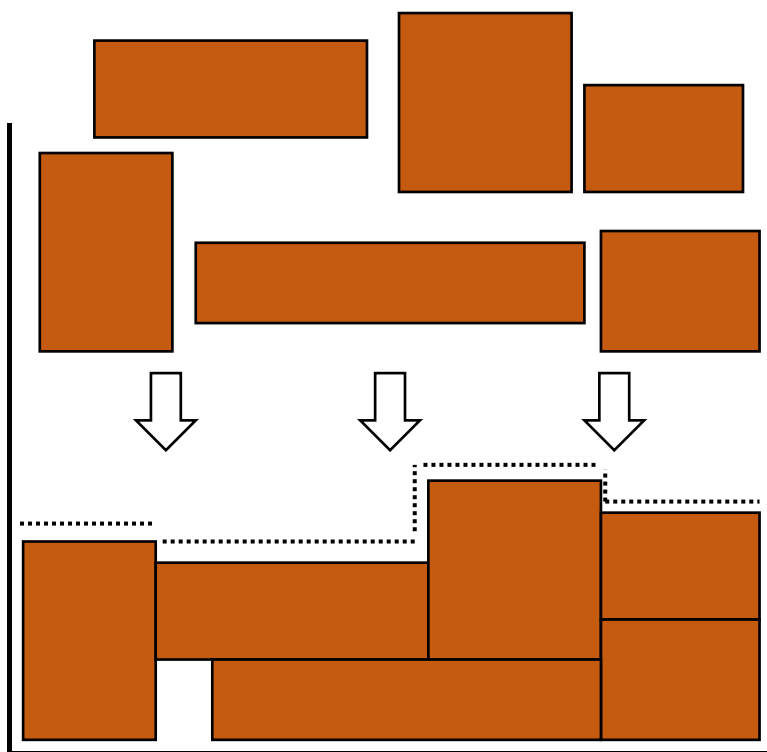
□ 缺点

- 对于一个包含 n 个模块的问题，即使是最简单的情况（不考虑模块旋转）
 - 需要 $2n$ 个连续变量(x_i, y_i)
 - 需要 $n(n - 1)$ 个整数变量(x_{ij}, y_{ij})
 - 需要 $2n(n - 1) + n(n - 1) + 4n$ 个线性约束，分别对应非重叠约束，整数变量的取值范围，以及模块放置的范围
- 所以实际上，这种方法只能解决小规模问题
- 并且，最终的结果质量受限于初始的约束条件（例如 w 的设置）

■ 线性规划

□ 对于较大规模的问题，可以使用逐步增强法

- 是一种贪心方法，用于保持问题规模较小，但并不能保证得到最优解：
 - 重复选择一个小的模块子集来构建混合整数线性规划，并将其与之前已选择且位置和形状固定的模块一起求解



下一个子集

部分解

■ 布图规划总结

□ Stockmeyer

- 使用给定模块集的切片结构
- 动态规划（只保留非冗余解）

□ 切片树

- 切片布图规划
- 波兰表达式

□ 序列对

- 非切片结构
- 简洁的紧凑表示

□ 混合整数线性规划

- 非切片结构
- 不可扩展

感谢！
